

Marketing Campaign & Event Data

Funnel Analysis

SQL Script for Pre Analysis

- In Our Database we have total 3 Tables
- 1. Events table
- 2. Campaigns table
- 3. Campaigns Spend table
- We have data of Month Jan, Feb, and Mar..
- We have 4 different Campaigns that are running in org, which renew after every 5th day and cost occur to our Org
- We will check for Duplicates and clean the data if required
 - We will create the reusable views, and do the per user analysis
- We will Analyse the Overall Campaign performance of our Campaigns
- We will do the Campaign Level Funnel Performance
- We will find the ROI and Cost per Conversion

Use CRMdata; **#Our Main Database**

```
CREATE TABLE Events (  
  User_id VARCHAR(100),  
  Event_time DATETIME,  
  Event_type CHAR(100),  
  Campaign_id VARCHAR(50),  
  Channel CHAR(100)  
); #Created Table to capture Events data
```

```
select * from Events;
```

```
CREATE TABLE Campaigns (  
  Campaign_id VARCHAR(100),  
  Campaign_name CHAR(100),  
  Channel CHAR(100),  
  Objective VARCHAR(50)  
); #Created table to capture Campaigns Data
```

```
CREATE TABLE Campaign_Spend (  
  Campaign_id VARCHAR(100),  
  Spend_date DATE,  
  Cost INT4  
); #Created table for capturing Campaign Spend Data
```

```
select * from Campaign_Spend;
```

Campaign_id	Spend_date	Cost
CAMP_1	24/01/2001	110
CAMP_1	24/01/2006	110
CAMP_1	24/01/2011	110
CAMP_1	24/01/2016	110
CAMP_1	24/01/2021	110
CAMP_1	24/01/2026	110
CAMP_1	24/01/2031	110
CAMP_1	24/02/2005	110
CAMP_1	24/02/2010	110
CAMP_1	24/02/2015	110
CAMP_1	24/02/2020	110
CAMP_1	24/02/2025	110
CAMP_1	24/03/2001	110
OK, 72 records retrieved in 3.919ms		

Create View Ordered_Events As

SELECT

User_id, Event_type, Event_time, Campaign_id, Channel,

ROW_NUMBER() over(partition by user_id, Event_type order by Event_time ASC)

as Row_Num

FROM Events;

#We have used this for eliminating Duplicate user with Same Event type,

#wherever we will find Row_Num>1 this means that user has repeated for the same event type

#Objectives we have fulfilled

#Removes duplicate events

#Keeps first occurrence per stage

#Prevents fake conversion inflation

Select * from Ordered_Events

User_id	Event_type	Event_time	Campaign_id	Channel	Row_Num
U0001	impression	09/02/2024	CAMP_1	Google Ads	1
U0002	impression	18/01/2024	CAMP_2	LinkedIn	1
U0003	impression	01/02/2024	CAMP_4	Google Ads	1
U0004	click	21/01/2024	CAMP_2	LinkedIn	1
U0004	converted	23/01/2024	CAMP_2	LinkedIn	1
U0004	impression	20/02/2024	CAMP_2	LinkedIn	1
U0004	lead	21/02/2024	CAMP_2	LinkedIn	1
U0004	qualified	22/01/2024	CAMP_2	LinkedIn	1
U0005	click	15/01/2024	CAMP_4	Google Ads	1
U0005	impression	15/01/2024	CAMP_4	Google Ads	1
U0006	click	09/01/2024	CAMP_4	Google Ads	1
U0006	impression	09/01/2024	CAMP_4	Google Ads	1
U0006	lead	10/01/2024	CAMP_4	Google Ads	1

#Lets check for Duplicates

```
select * from Ordered_Events
```

```
where Row_Num>1;
```

#No Data Displayed means no duplication is there

#Creating this view to analyse the event type per user and also to seek information about campaign (from which the user has come in)

Create View User_Funnel as

Select User_id, Campaign_id,

Max(case when Event_type="impression" then 1 else 0 END) as Impression,

Max(case when Event_type="click" then 1 else 0 END) as Click,

Max(case when Event_type="lead" then 1 else 0 END) as Lead_y,

Max(case when Event_type="qualified" then 1 else 0 end) as Qualified,

Max(case when Event_type="converted" then 1 else 0 end) as Converted

from Ordered_Events

where Row_Num=1

GROUP BY user_id, campaign_id;

Select * From User_Funnel;

User_id	Campaign_id	Impression	Click	Lead_y	Qualified	Converted
U0001	CAMP_1	1	0	0	0	0
U0002	CAMP_2	1	0	0	0	0
U0003	CAMP_4	1	0	0	0	0
U0004	CAMP_2	1	1	1	1	1
U0005	CAMP_4	1	1	0	0	0
U0006	CAMP_4	1	1	1	0	0
U0007	CAMP_1	1	1	1	1	0
U0008	CAMP_4	1	1	1	1	1
U0009	CAMP_3	1	1	1	1	1
U0010	CAMP_2	1	0	0	0	0
U0011	CAMP_3	1	1	0	0	0
U0012	CAMP_3	1	0	0	0	0
U0013	CAMP_2	1	1	0	0	0

-- Funnel Count and Conversion Rates

```
Select Count(*) as Total_Users,
sum(Impression) as Total_Impressions,
sum(Click) as Clicked_Users,
sum(Lead_y) as Total_Leads,
sum(Qualified) as Qualified_Leads,
sum(Converted) as Conversions,
ROUND(1.0 * SUM(converted) / COUNT(*), 2)*100 AS Overall_Conversion_Rate
From User_Funnel;
```

Total_Users	Total_Impressions	Clicked_Users	Total_Leads	Qualified_Leads	Conversions	Overall_Conversion_Rate
150	150	121	90	60	27	18.00

OK, 1 record retrieved in 85.574ms

View: Pages: Edit:

-- Campaign Level Funnel Performance

```
SELECT
Campaign_id,
count(*) as Users,
sum(Converted) as Conversions,
ROUND(1.0 * SUM(converted) / COUNT(*), 2)*100 AS Conversion_Rate
FROM User_Funnel
GROUP BY Campaign_id;
```

Campaign_id	Users	Conversions	Conversion_Rate
CAMP_1	33	8	24.00
CAMP_2	42	5	12.00
CAMP_4	39	5	13.00
CAMP_3	36	9	25.00

OK, 4 records retrieved in 91.666ms

-- ROI & Cost per Conversion

```
WITH campaign_conversions AS (
  SELECT
    campaign_id,
    SUM(converted) AS conversions
  FROM user_funnel
  GROUP BY campaign_id
),
campaign_cost AS (
  SELECT
```

```

        campaign_id,
        SUM(cost) AS total_cost
    FROM campaign_spend
    GROUP BY campaign_id
)
SELECT
    c.campaign_id,
    total_cost,
    conversions,
    ROUND(total_cost / NULLIF(conversions, 0), 2) AS cost_per_conversion
FROM campaign_cost c
LEFT JOIN campaign_conversions cc
ON c.campaign_id = cc.campaign_id;

```

campaign_id	total_cost	conversions	cost_per_conversion
CAMP_1	1980	8	247.50
CAMP_2	3600	5	720.00
CAMP_3	2700	9	300.00
CAMP_4	1800	5	360.00

OK, 4 records retrieved in 29.281ms